One Page.
All Concepts.
Revise Fast.
Score High!

ULTIMATE REVISION

ENCRYPTION + JWT

Smart Revision Better Results

CRACK INTERVIEWS & EXAMS

QUICK REVISION NOTES

ENCRYPTION

Symmetric & Asymmetric
AES, RSA, ECC

Algorithms
AES, DES, 3DES, RSA, ECC

Keys & Key Management
Types, Best Practices

Hashing
SHA-256, SHA-512, Bcrypt, Argon2

Security Best Practices
Confidentiality, Integrity, Authentication



JWT (JSON WEB TOKEN)

Structure & Components
Header, Payload, Signature

Claims
Standard, Public, Private Claims

Signing & Verification
Algorithms, Secret / Public Key

Authentication Flow
Login → Token → Access → Verify

Security Best Practices
Short-lived Tokens, HTTPS, Storage, Revocation

ENCODING vs ENCRYPTION vs HASHING

ENCODING	ENCRYPTION	HASHING
• Data Representation	• Confidentiality	• Data Integrity
• Human Readable	• Not Human Readable	• Not Reversible (One-way)
• Reversible	• Reversible (with key)	• Fixed Size Output
• Example: Base64	• Example: AES, RSA	• Example: SHA-256

REMEMBER

- Encrypt to Keep Secret
- Hash to Ensure Integrity
- JWT to Verify & Share Claims

> "Secure the data. Verify the user. Trust nothing, verify everything."

JWT AUTHENTICATION FLOW (HIGH LEVEL)

1. **USER LOGIN** — User provides credentials
2. **SERVER AUTHENTICATES & RETURNS JWT** — Access Token + Refresh Token
3. **CLIENT STORES TOKEN SECURELY** — Memory / HttpOnly Cookie
4. **CLIENT SENDS TOKEN IN REQUESTS** — Authorization: Bearer <token>
5. **SERVER VALIDATES TOKEN** — Check signature, claims, expiry
6. **SERVER SENDS RESPONSE** — Protected data returned

ONE PAGE POWER
All important concepts at a glance.

QUICK REVISION
Save time. Revise smart.

EXAM & INTERVIEW FOCUSED
High weightage topics covered.

SCORE HIGH CONFIDENTLY
Be prepared. Be unbeatable.

SAVE THIS POST | SHARE WITH FRIENDS | LIKE IF IT HELPED YOU!

1. Encryption & Decryption Fundamentals

- **What is Encryption?**

  Encryption is the process of converting readable data (plaintext) into unreadable format (ciphertext) using an algorithm and a key. It ensures confidentiality of data.

- **What is Decryption?**

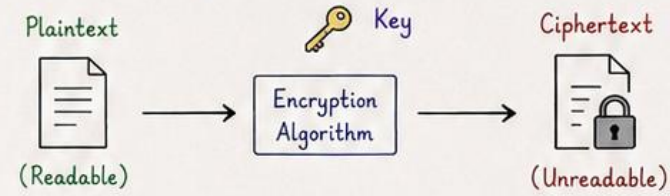
  Decryption is the reverse process of encryption. It converts ciphertext back to original plaintext using an algorithm and a key.

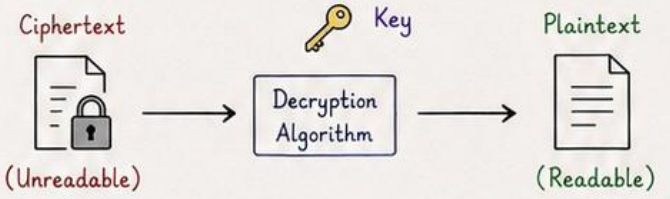
- **Why do we need Encryption?**
  - Protect data from unauthorized access
  - Secure data during storage and transmission
  - Ensure privacy and confidentiality
  - Maintain data integrity and authenticity

- **Basic Terminology**

Plaintext	Original readable data.
Ciphertext	Encrypted (unreadable) data.
Algorithm	Mathematical steps used for encryption/decryption.
Key	Secret value used by the algorithm.
Encryption	Process of converting plaintext to ciphertext.
Decryption	Process of converting ciphertext back to plaintext.

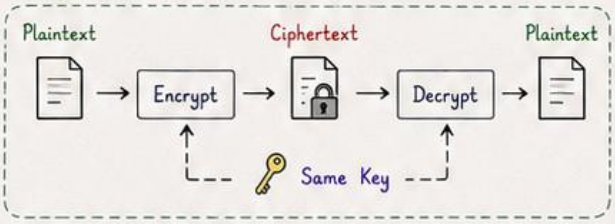
- **How Encryption Works?**



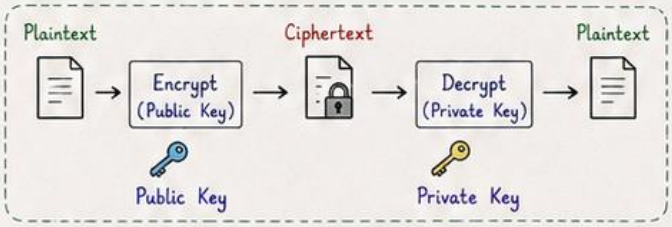


- **Types of Encryption**

1. Symmetric Encryption
   - Same key is used for both encryption and decryption.
   - Fast and efficient.
   - Example: AES, DES



2. Asymmetric Encryption
   - Uses a pair of keys: Public Key and Private Key.
   - Public key for encryption, private key for decryption (or vice-versa).
   - Slower but more secure.
   - Example: RSA, ECC



- **Key Takeaways**

✓ Encryption converts readable data into unreadable form.
✓ Decryption restores the original data.
✓ Keys and algorithms are core components of security.
✓ Encryption ensures confidentiality, privacy and trust.

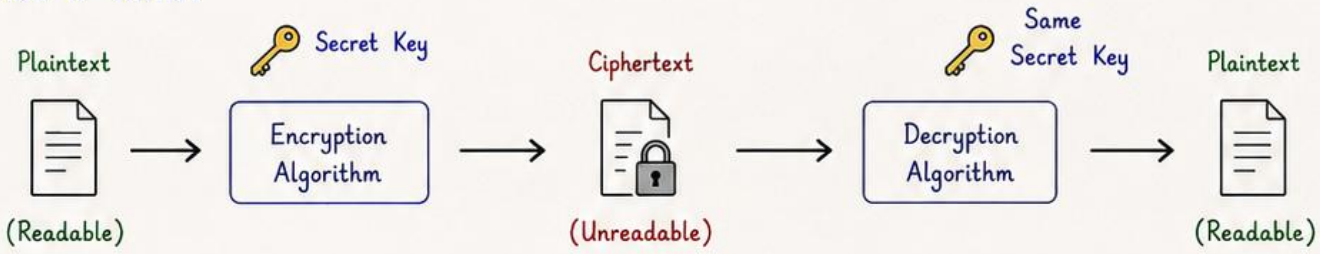
Note :
Encryption != Hashing
Encryption is reversible.
Hashing is one-way.
Do not try to reverse hashed data.

2. Symmetric Encryption

What is Symmetric Encryption?

Symmetric Encryption uses the same secret key for both encryption and decryption.
The sender and receiver must share this key securely.

How it Works?



Characteristics

- Uses a single shared secret key.
- Fast and efficient.
- Suitable for large amount of data.
- Key distribution is critical.
- If the key is compromised, data is at risk.

Advantages

- ✓ Faster than asymmetric encryption.
- ✓ Efficient for bulk data encryption.
- ✓ Uses simpler algorithms.

Disadvantages

- ✗ Key distribution is difficult and less secure.
- ✗ If the key is leaked, all data is at risk.
- ✗ Not suitable for large number of users.

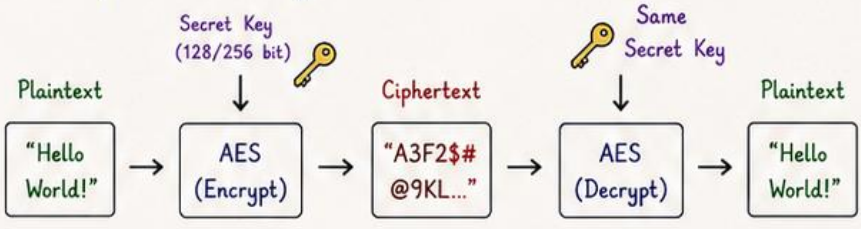
Common Symmetric Algorithms

Algorithm	Key Size	Block Size	Description	Use Cases
AES (Advanced Encryption Standard)	128, 192, 256 bits	128 bits	Most widely used symmetric algorithm. Secure and efficient.	Data encryption, VPN, SSL/TLS
DES (Data Encryption Standard)	56 bits	64 bits	Older standard, now considered less secure.	Legacy systems (avoid now)
3DES (Triple DES)	112, 168 bits	64 bits	Applies DES algorithm three times for better security.	Banking systems, legacy support
Blowfish	32 – 448 bits	64 bits	Fast and secure. Used in many applications.	File encryption, Software
ChaCha20	256 bits	–	Modern algorithm, fast in software and secure.	Mobile, VPN, TLS

Key Points

- ✓ The strength of symmetric encryption depends on the key size and algorithm.
- ✓ Longer key = more security.
- ✓ Keep the key secret and safe.
- ✓ Use strong algorithms like AES.

Example: AES Encryption



Note :

- Symmetric encryption is 20–100 times faster than asymmetric encryption.
- It is widely used in real-world applications for performance and security.
- Always ensure secure key generation, storage and sharing.



3. Asymmetric Encryption

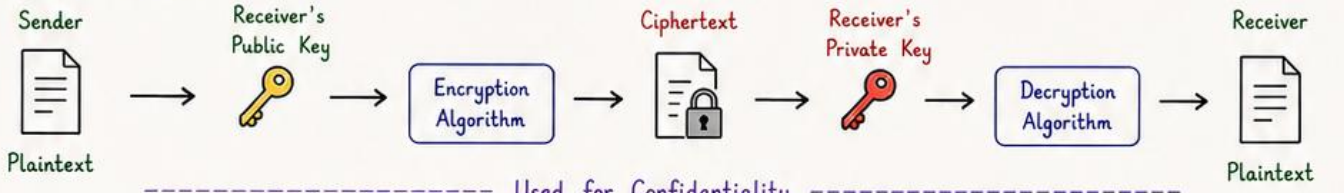
What is Asymmetric Encryption?

Asymmetric Encryption (Public Key Encryption) uses A pair of keys: a Public Key and a Private Key. One key is used to encrypt data and the other key is used to decrypt it.
The keys are mathematically related but cannot be derived from each other.

Key Idea

- ✓ Public Key → can be shared (no harm)
- ✓ Private Key → must be kept secret at all times

How it Works?





Characteristics

- Uses a pair of keys: Public Key and Private Key.
- Public Key can be shared openly.
- Private Key must be kept secret.
- Slower than symmetric encryption.
- More secure for key distribution.
- Supports encryption, decryption, digital signatures and key exchange.

Use Cases

- Secure data transmission over the internet (SSL/TLS)
- Email encryption (PGP)
- Digital signatures
- Authentication
- Secure key exchange

Example: RSA, ECC (Elliptic Curve Cryptography)

Common Asymmetric Algorithms

Algorithm	Key Size	Security Level	Description	Use Cases
RSA	1024, 2048, 4096 bits	High	Most widely used asymmetric algorithm based on factoring large numbers.	SSL/TLS, Email Encryption, Digital Signatures
ECC (Elliptic Curve Cryptography)	256, 384, 521 bits	Very High	Provides same security with smaller key sizes. More efficient than RSA.	Mobile security, IoT, SSL/TLS, Digital Signatures
ElGamal	1024, 2048 bits	High	Based on discrete logarithm problem.	Encryption, Key Exchange
DSA (Digital Signature Algorithm)	1024, 2048 bits	High	Used only for digital signatures.	Digital Signatures

Key Points

- ✓ Provides better security for key distribution.
- ✓ Slower in performance compared to symmetric encryption.
- ✓ Public key can be shared, private key must be secret.
- ✓ Used for both encryption and digital signatures.
- ✓ Greater computational overhead.

Key Pair Generation (Example: RSA)

1. Choose two large prime numbers (p and q)
2. Compute $n = p \times q$
3. Compute totient $\varphi(n) = (p-1) \times (q-1)$
4. Choose e such that $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$
5. Compute d such that $(d \times e) \bmod \varphi(n) = 1$
6. Public Key = (e, n)
7. Private Key = (d, n)

Public Key (e, n)

Private Key (d, n)

Note : Asymmetric encryption is the foundation of modern secure communication on the internet.

4. Hashing + Hash Functions

What is Hashing?

Hashing is the process of converting input data of any size into a fixed-size string of characters using a mathematical algorithm called a hash function. The output is called a hash value or message digest.

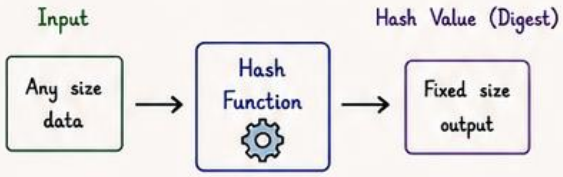
Analogy



Hashing is like a blender.

- ✓ No matter how much or what you put in,
- ✓ You always get output of same size (smoothie).
- ✓ You cannot take the smoothie and get the original fruits back.

How Hashing Works?



- One-way function
- Fast to compute
- Very hard to reverse
- Small change in input → large change in hash

Hashing vs Encryption

Hashing	Encryption
One-way (irreversible)	Two-way (reversible)
Fixed size output	Variable size output
No key required	Key is required
Used for integrity	Used for confidentiality

Common Hash Functions

Algorithm	Hash Size	Description	Use Cases
MD5 (Message Digest 5)	128 bits	– Old and broken (collision attacks possible). – Not recommended for security.	Legacy systems, checksums (avoid for security)
SHA-1 (Secure Hash Algorithm 1)	160 bits	– Also considered weak now. – Deprecated for most security uses.	Legacy systems (avoid for security)
SHA-256 (Secure Hash Algorithm 256-bit)	256 bits	– Part of SHA-2 family. – Strong and widely used.	SSL/TLS, Blockchain, Digital Signatures
SHA-512 (Secure Hash Algorithm 512-bit)	512 bits	– Part of SHA-2 family. – More secure but slower than SHA-256.	High security applications, Digital Signatures
SHA-3 (Secure Hash Algorithm 3)	Variable (224/256/384/512 bits)	– Latest standard (Keccak algorithm). – Strong against future attacks.	Future-proofing, Cryptographic applications

Properties of a Good Hash Function

- ✓ Deterministic – Same input always gives same hash.
- ✓ Fast – Quick to compute.
- ✓ Fixed-size output – Output size is constant.
- ✓ Pre-image resistance – Given hash, it should be computationally infeasible to find the input.
- ✓ Second pre-image resistance – Given an input, it should be infeasible to find another input with the same hash.
- ✓ Collision resistance – It should be infeasible to find any two inputs with the same hash.

Example

Input	SHA-256 Hash (Hex)
"hello"	2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e9f4891c96bfaad9b6e991b7852
"Hello"	185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
"hello!"	7596e4f7e4d7b0f6c2bba8547c6d1c6d5f1a0b2a7b6f0b3d7c6b5a9e6d3f1c0a2b3d4e5f6071829

➡ Small change in input → completely different hash.

Where is Hashing Used?

- Password Storage (never store plain passwords).
- Data Integrity (checksums, file verification).
- Digital Signatures.
- Blockchain (block linking).
- Message Authentication (HMAC).

Important Note

Hashing is not encryption.
You cannot decrypt or reverse a hash.
Always use strong hash functions like SHA-256, SHA-512, SHA-3 for security.



⭐ Remember: Hashing ensures data integrity, while Encryption ensures data confidentiality.

5. Salt + Password Hashing

What is Salt?

A salt is a random value added to a password before hashing.
It ensures that even if two users have the same password, their hashes will be different.

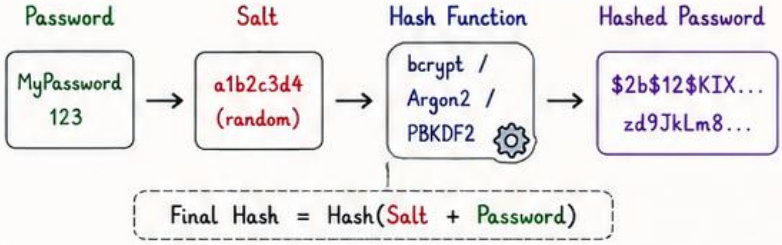
Rainbow Table Attack

Attackers use precomputed tables of hash values for common passwords.
If no salt is used, attackers can easily find the original password by matching the hash.

Why Salt is Important?

- ✓ Prevents rainbow table attacks.
- ✓ Ensures same password has different hash for different users.
- ✓ Adds randomness and uniqueness.
- ✓ Strongly increases security.

How it Works?



Types of Salt

Type	Description
Unique Salt	A unique random salt for each user. (Recommended)
Global Salt	Same salt for all users. (Not Recommended)
No Salt	No salt used. (Highly insecure)

Best Practices

- ✓ Use a unique, long, and random salt for each user.
- ✓ Store the salt with the hashed password.
- ✓ Use a slow and strong hashing algorithm.
- ✓ Never store plain passwords.
- ✓ Use additional techniques like Pepper if required.

Pepper: A secret value (stored in server/config) added along with salt for extra security.

Common Password Hashing Algorithms

Algorithm	Type	Features	Use Cases
bcrypt	Adaptive (Blowfish based)	- Built-in salt - Adjustable cost factor (work factor) - Slower = more secure	Web applications, Login systems
Argon2	Memory-hard	- Winner of Password Hashing Competition - Resistant to GPU, ASIC attacks - Configurable memory, time, parallelism	High security systems, Modern applications
PBKDF2	Key Derivation Function	- Uses HMAC (SHA-256/512) - Configurable iterations	Legacy systems, APIs, Cryptographic storage

Example (bcrypt)

1. Password : MyPassword123
2. Generate Salt : $2b$12$KIX... (random salt generated by bcrypt)
3. Hash : $2b$12$KIXzd9JkLm8Qz... (final hash)

```
// Node.js example using bcrypt
const bcrypt = require('bcrypt');

const saltRounds = 12;
const hash = await bcrypt.hash('MyPassword123', saltRounds);
console.log(hash);
// $2b$12$KIXzd9JkLm8Qz...
```

How to Store?



id	salt	hash
1	a1b2c3d4...	$2b$12$KIX...
...	...	...

Note: In many modern algorithms like bcrypt, salt is already included in the hash string. So you may only need to store the hash.

Key Takeaways

- ✓ Always use a unique salt for each user.
- ✓ Salt protects against rainbow table attacks.
- ✓ Use strong hashing algorithms like bcrypt or Argon2.
- ✓ Never store plain passwords.

Remember:

★ Salt + Strong Hashing = Strong Security

6. AES + Common Encryption Algorithms

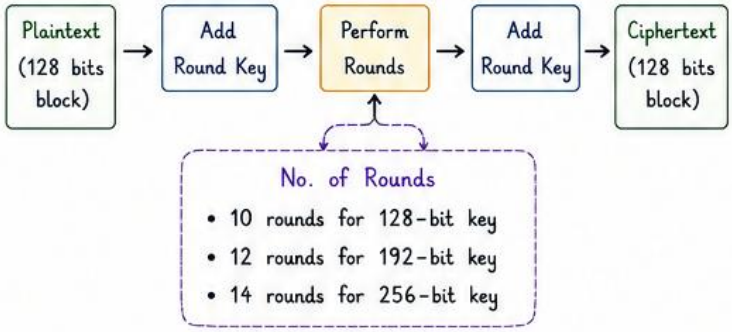
What is AES?

AES (Advanced Encryption Standard) is a symmetric block cipher algorithm used to encrypt and decrypt data. It is secure, fast and widely used across the world. It uses the same secret key for both encryption and decryption.

Key Points

- ✓ AES is a symmetric key algorithm.
- ✓ It works on fixed block size of 128 bits.
- ✓ Key sizes can be 128, 192 or 256 bits.
- ✓ It is secure, efficient and widely adopted.

How AES Works? (High Level)



AES Round Operations

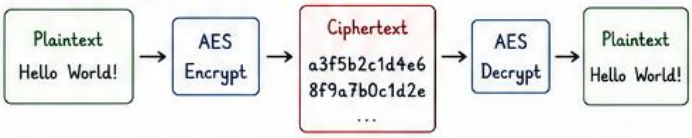
1	SubBytes	Substitution using S-Box (non-linear step)
2	ShiftRows	Shift rows of the state array
3	MixColumns	Mix data within each column (except last round)
4	AddRoundKey	XOR the state with round key

Note: In the final round, MixColumns is not performed.

Common Encryption Algorithms

Algorithm	Type	Key Size	Block Size	Description	Use Cases
AES (Advanced Encryption Standard)	Symmetric (Block Cipher)	128, 192, 256 bits	128 bits	• Most widely used algorithm. • Secure and efficient. • Standard for modern encryption.	SSL/TLS, VPN, Wi-Fi Security, File Encryption
DES (Data Encryption Standard)	Symmetric (Block Cipher)	56 bits	64 bits	• Older standard, now considered insecure due to small key size.	Legacy systems (Avoid in new applications)
3DES (Triple DES)	Symmetric (Block Cipher)	112, 168 bits	64 bits	• Applies DES algorithm three times. • More secure than DES. • Slower than AES.	Banking systems, Legacy applications
Blowfish	Symmetric (Block Cipher)	32 – 448 bits (variable)	64 bits	• Fast and secure. • Variable key size.	File encryption, Software
Twofish	Symmetric (Block Cipher)	128, 192, 256 bits	128 bits	• Designed as AES finalist. • Fast and secure.	Disk encryption, Software
ChaCha20	Symmetric (Stream Cipher)	256 bits	–	• Modern stream cipher. • Very fast in software. • Used in TLS, VPN.	Mobile, VPN, TLS, SSH
RC4 (Rivest Cipher 4)	Symmetric (Stream Cipher)	40 – 2048 bits	–	• Fast but insecure now. • Deprecated.	Legacy systems (Avoid now)

AES Example (128-bit Key)



Example (Using OpenSSL)

```
# Encrypt
openssl enc -aes-128-cbc -in file.txt -out enc.txt -k mypassword

# Decrypt
openssl enc -aes-128-cbc -d -in enc.txt -out dec.txt -k mypassword
```

Symmetric vs Asymmetric (Quick Comparison)

Feature	Symmetric	Asymmetric
Keys	Same secret key	Public & Private key pair
Speed	Faster	Slower
Key Size	Smaller (128/256 bits)	Larger (1024/2048 bits)
Use Case	Bulk data encryption	Key exchange, Digital signatures
Examples	AES, DES, 3DES, Blowfish, RC4	RSA, ECC, ElGamal

Key Takeaways

- ✓ AES is the gold standard for symmetric encryption.
- ✓ Always prefer AES-256 for maximum security.
- ✓ Use strong keys and secure key management.
- ✓ Avoid older algorithms like DES and RC4.

Important Note

★ The security of encryption depends not only on the algorithm but also on key strength, implementation and proper usage.

7. RSA + Public Key Cryptography

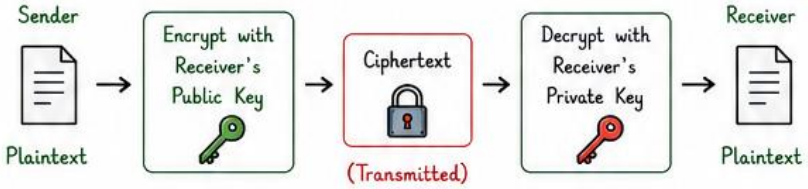
What is RSA?

RSA (Rivest–Shamir–Adleman) is the most widely used asymmetric encryption algorithm.
It uses a pair of keys: a Public Key (to encrypt) and a Private Key (to decrypt).
It is based on the mathematical hardness of factoring large prime numbers.

Key Points

- ✓ Uses two different but mathematically related keys.
- ✓ Public Key can be shared openly.
- ✓ Private Key must be kept secret.
- ✓ Slower than symmetric encryption.
- ✓ Used for secure key exchange, digital signatures and encrypting small data.

How RSA Works?



For Digital Signatures: Sender signs with Private Key and Receiver verifies with Public Key.

RSA Key Generation (High Level)

1. Choose two large prime numbers (p and q)
2. Compute $n = p \times q$
3. Compute $\varphi(n) = (p - 1) \times (q - 1)$
4. Choose e such that $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$
5. Compute d such that $(d \times e) \bmod \varphi(n) = 1$
6. Public Key = (e, n)
7. Private Key = (d, n)

RSA Key Components

Component	Description
Public Key (e, n)	• e is the public exponent • n is the modulus (product of two large primes) • This key is shared with everyone
Private Key (d, n)	• d is the private exponent • n is the same modulus • This key must be kept secret
n (Modulus)	Product of two large prime numbers (p × q)
φ(n) (Totient)	(p − 1) × (q − 1) – used in key generation

RSA Encryption & Decryption

Operation	Formula	
Encryption	$C = M^e \bmod n$	M = Message (as integer) C = Ciphertext e = Public exponent n = Modulus
Decryption	$M = C^d \bmod n$	d = Private exponent n = Modulus

Note: Power and mod operations are performed on very large numbers.

Example (Small Numbers for Understanding)

1. Choose p and q	2. Compute n	3. Compute φ(n)	4. Choose e	5. Compute d	6. Keys
p = 61, q = 53	n = p × q = 61 × 53 = 3233	φ(n) = (p − 1)(q − 1) = 60 × 52 = 3120	Let e = 17 (gcd (17, 3120) = 1)	Find d such that (d × 17) mod 3120 = 1 d = 2753	Public Key (e, n) = (17, 3233) Private Key (d, n) = (2753, 3233)

Encryption

Let message M = 65

$C = 65^{17} \bmod 3233$

$= 2790$

Decryption

$M = 2790^{2753} \bmod 3233$

$= 65$

Original message recovered!

Uses of RSA

- ✓ Secure key exchange
- ✓ Digital signatures
- ✓ Encrypting small data (e.g., session keys)
- ✓ SSL/TLS handshakes
- ✓ Email encryption (S/MIME, PGP)

RSA Limitations

- ✗ Slower than symmetric algorithms
- ✗ Not suitable for large data
- ✗ Requires large key sizes for security

RSA Key Sizes & Security (General Guideline)

Key Size (bits)	Security Level (Approx.)
1024	Deprecated (Insecure)
2048	~112 bits (Good)
3072	~128 bits (Better)
4096	~152 bits (Very Strong)

Common RSA Padding Schemes

- PKCS#1 v1.5 – Older padding (encryption)
- OAEP (Optimal Asymmetric Encryption Padding) – Recommended
- PSS (Probabilistic Signature Scheme) – For digital signatures

RSA vs Symmetric (Quick Comparison)

Feature	RSA (Asymmetric)	Symmetric (e.g., AES)
Keys	Public + Private	One secret key
Speed	Slow	Fast
Key Size	Large (2048+ bits)	Small (128/256 bits)
Best Use	Key exchange, signatures, small data	Bulk data encryption

Key Takeaways

- ✓ RSA uses a pair of keys: one public, one private.
- ✓ Security is based on the difficulty of factoring large numbers.
- ✓ Public Key can be shared; Private Key must be kept secret.
- ✓ Use RSA for small data, not for large files.

Important Note

Always use strong key sizes (2048 bits or more) and secure padding (OAEP for encryption, PSS for signatures).

8. Digital Signatures

What is a Digital Signature?

A digital signature is a cryptographic technique used to verify the authenticity, integrity and non-repudiation of digital data or messages.

It is the digital equivalent of a handwritten signature.

Key Points

- ✓ Uses asymmetric cryptography (public/private key pair).
- ✓ Signer signs with private key.
- ✓ Anyone can verify with the signer's public key.
- ✓ Provides authenticity, integrity and non-repudiation.
- ✓ Widely used in SSL/TLS, code signing, documents, blockchain, emails, e-governance, etc.

Why Digital Signatures?

- ✓ Ensures authenticity (verifies the sender)
- ✓ Ensures integrity (data not tampered)
- ✓ Provides non-repudiation (sender cannot deny)
- ✓ Works over insecure channels (Internet, email, etc.)

How Digital Signatures Work?

Verifier (Receiver)

1. **Message** — Data
2. **Hash Function** (e.g., SHA-256) — Produces fixed size hash (digest)
3. **Sign with Private Key** — Encrypt the hash using private key
4. **Digital Signature** — Signature is attached to the original message
5. **Received Message** — Message + Signature
6. **Hash Function** (same as step 2) — Hash the received message again
7. **Verify with Public Key** — Decrypt the signature using signer's public key
8. **Verification Result** — If both hashes are same ⇒ Valid Signature Else ⇒ Invalid

Components of Digital Signature

Component	Description
Message (M)	Original data to be signed.
Hash Function (H)	Converts message into fixed-size hash (digest). Examples: SHA-256, SHA-512
Private Key (d)	Used by the signer to sign the hash.
Public Key (e)	Used by anyone to verify the signature.
Digital Signature (S)	Encrypted hash using signer's private key. $S = H(M)^d \bmod n$

Digital Signature Process (Formulas)

Process	Formula
Signing	1. $h = H(M)$ (Hash of message) 2. $S = h^d \bmod n$ (Sign using private key)
Verification	1. $h_1 = H(M)$ (Hash of received message) 2. $h_2 = S^e \bmod n$ (Decrypt signature using public key) 3. If $h_1 == h_2$ ⇒ Valid else ⇒ Invalid

Note: d = Private exponent, e = Public exponent, n = Modulus (p × q), H = Hash function

Common Digital Signature Algorithms

Algorithm	Description	Key Size	Hash Used	Use Cases
RSA	Most widely used public key algorithm for digital signatures.	1024, 2048, 3072, 4096+ bits	SHA-256 / SHA-512	SSL/TLS, Code signing, PDF signing, Emails
DSA	Digital Signature Algorithm (NIST Standard)	1024 (L=160), 2048, 3072 bits	SHA-1 (old), SHA-256	Government, legacy systems
ECDSA	Elliptic Curve Digital Signature Algorithm	160, 224, 256, 384, 521 bits	SHA-256 / SHA-512	Mobile, IoT, Blockchain, Modern applications
EdDSA (Ed25519)	Modern, fast and secure signature algorithm	256 bits	SHA-512	Crypto libraries, modern protocols

Example (RSA Signature)

Suppose:
Message (M) = "Hello World"
Hash function = SHA-256
Private Key (d, n) and Public Key (e, n)

Signing (by sender)
1. h = SHA-256("Hello World")
2. $S = h^d \bmod n$ → Signature

Verification (by receiver)
1. h_1 = SHA-256("Hello World")
2. $h_2 = S^e \bmod n$
3. If $h_1 == h_2$ ⇒ Valid Signature else ⇒ Invalid Signature

Uses of Digital Signatures

- SSL/TLS certificates (HTTPS security)
- Code signing (software, executables)
- Digitally signed documents (PDF, e-forms)
- Email security (S/MIME, PGP)
- Blockchain & smart contracts
- E-Governance, legal documents, tenders

Important Note

Digital signatures provide more than just confidentiality. They provide trust, integrity and authenticity.

Digital Signature vs Encryption (Quick Comparison)

Feature	Digital Signature	Encryption
Goal	Authentication, Integrity, Non-repudiation	Confidentiality
Keys Used	Private key (sign), Public key (verify)	Public key (encrypt), Private key (decrypt)
Output	Signature (small)	Encrypted data (large)
Provides	Trust & authenticity	Secrecy of data

Key Takeaways

- ✓ Digital Signatures use asymmetric cryptography.
- ✓ Signer uses private key; anyone verifies using public key.
- ✓ Ensures authenticity, integrity and non-repudiation.
- ✓ Widely used in real-world security.

9. Keys + Key Management



What are Keys?

Keys are secret values used in cryptography to encrypt, decrypt, sign or verify data.
Without the correct key, the data remains unreadable or can't be verified.

Key Points

- ✓ Keys are just numbers (bits).
- ✓ They control access to encrypted data.
- ✓ Keys must be kept secret and managed properly.
- ✓ Key management is as important as encryption.
- ✓ Compromised keys can lead to data breaches.



Types of Keys

Type	Purpose	Examples
Encryption Key	Used to encrypt data (also used for decryption in symmetric encryption).	AES key
Decryption Key	Used to decrypt data (usually same as encryption key in symmetric).	AES key (shared)
Public Key	Used to encrypt data or verify signatures.	RSA public key
Private Key	Used to decrypt data or create digital signatures.	RSA private key
Session Key	Temporary key for a session (e.g., TLS).	TLS session key

Key Pairs (Asymmetric Encryption)

Public Key (Shared)



Private Key (Kept Secret)



Remember

In a key pair, public key is for encrypt/verify, private key is for decrypt/sign.

- Public key can be shared with everyone.
- Private key must be kept secret.
- Used for encryption, decryption, and digital signatures.

Key Lifecycle

Generate → Store → Use → Rotate → Revoke

- Generate: Create strong random keys.
- Store: Keep in secure storage (KMS/HSM).
- Use: Use only when needed.
- Rotate: Change keys periodically.
- Revoke: Invalidate compromised or unused keys.

Key Management Systems

Component	Description	Example
KMS	Manages, stores and rotates keys securely.	AWS KMS, Azure Key Vault
HSM	Hardware device to store keys (very secure).	YubiHSM, Thales
Secrets Manager	Stores application secrets (API keys, database creds, etc).	AWS Secrets Manager, HashiCorp Vault
Key Rotation	Automatically changes keys at regular intervals.	AWS KMS, Google Cloud KMS

Key Storage Methods

Method	Use Case	Pros / Cons
In Memory	Temporary keys (e.g., session keys)	Fast, but lost on restart.
File System	Local storage (not recommended)	Easy, but risky (if compromised).
Environment Variables	Simple apps	Convenient, but can be leaked.
KMS / HSM	Production, high security	Most secure, scalable, managed.

Key Management Best Practices

- ✓ Use strong, random key generation.
- ✓ Never hardcode keys in code.
- ✓ Store keys in secure vaults (KMS / HSM).
- ✓ Implement key rotation and expiration.
- ✓ Use least privilege access (only authorized services).
- ✓ Monitor and audit key usage.

Examples

1. Symmetric Key (AES)

- Key: 128 / 192 / 256 bits
- Usage: Encrypt and decrypt data.



(Same key for both)

2. RSA Key Pair

- Public Key: Encrypt / Verify
- Private Key: Decrypt / Sign
- Key Size: 2048 / 4096 bits



(Different keys)

3. TLS Session Key

- Temporary key used to secure communication between client and server.



(Rotated frequently)

Common Key Management Challenges

- ⚠ Key leakage (accidental or malicious).
- ⚠ Poor key rotation policies.
- ⚠ Over-privileged access.
- ⚠ Lost keys (no backup).
- ⚠ Untrusted storage.

Real-World Use Cases

- Cloud (AWS KMS, Azure Key Vault)
- Databases (TDE, encryption at rest)
- Web Security (TLS/SSL)
- Mobile Apps (secure storage)
- Digital Signatures (document verification)

Note :

Key management is not just about storing keys, it's about keeping them safe, accessible only to authorized users, and managing their lifecycle.

10. Encoding vs Encryption vs Hashing

Why These Concepts?

Encoding, Encryption and Hashing are often confused, but they solve different problems. Understanding the difference is essential for designing secure systems.

Key Points

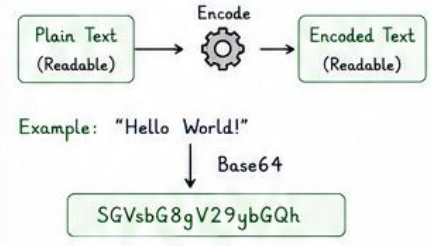
- ✓ Encoding is about representation (readable ↔ readable).
- ✓ Encryption is about confidentiality (readable ↔ unreadable).
- ✓ Hashing is about integrity (any size → fixed size).
- ✓ Use the right one for the right purpose!

Encoding vs Encryption vs Hashing (At a Glance)

Feature	Encoding	Encryption	Hashing
Purpose	Data representation / format conversion	Confidentiality – protect data from unauthorized access	Integrity – detect any change in data
Input	Any readable data (text, binary, etc.)	Any readable data	Any data
Output	Readable data	Unreadable (ciphertext)	Fixed-size string (digest)
Reversibility	Yes, always reversible	Yes, reversible with correct key	No, one-way (irreversible)
Key Used	No key	Yes, uses a key (symmetric or asymmetric)	No key
Same Input, Same Output	Yes	Not always (depends on key & algorithm)	Always Yes
If output is changed	Original can be recovered	Original can be recovered with correct key	Change is detected (different hash)
Main Use Cases	Data transmission, storing in safe character sets	Secure communication, data at rest, APIs, etc.	Password storage, integrity checks, digital signatures
Examples	Base64, URL Encoding, UTF-8, ASCII	AES, DES, RSA	SHA-256, SHA-1, MD5

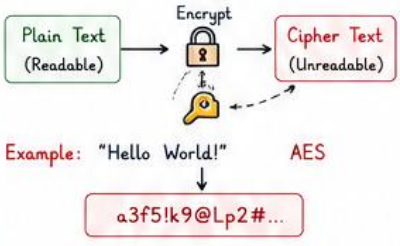
How They Work? (Conceptual View)

1. Encoding



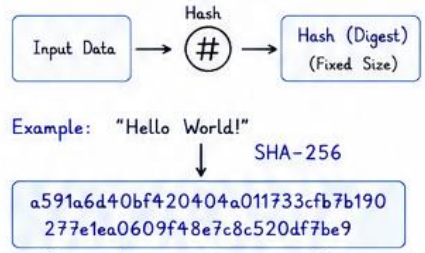
(Can be decoded back to original)

2. Encryption



(Can be decrypted back with correct key)

3. Hashing



(Cannot be reversed to original)

Common Use Cases

	Encoding	Encryption	Hashing
Web Transmission	URL encoding, Base64	HTTPS (TLS)	—
Data at Rest	—	AES, TDE	—
Password Storage	—	—	bcrypt, Argon2, SHA-256
Integrity Check	—	—	SHA-256, SHA-3
Digital Signatures	—	RSA (Sign/Verify)	Hash + Sign
Email Systems	Base64 (MIME)	PGP, S/MIME	SHA-256
Data Formats	UTF-8, ASCII, Base64	—	—

Important Notes

- ✓ Encoding is NOT security.
- ✓ Encryption protects confidentiality.
- ✓ Hashing ensures integrity, not confidentiality.
- ✓ Never use encryption when hashing is enough (and vice versa).
- ✓ For passwords, always use slow hash functions (bcrypt, Argon2).

Quick Comparison (Summary)

Aspect	Encoding	Encryption	Hashing
Goal	Representation	Confidentiality	Integrity
Key	No	Yes	No
Output	Readable	Unreadable	Fixed Size
Reversible	Yes	Yes (with key)	No
Detect Changes	No	No (needs decrypt)	Yes
Typical Examples	Base64, UTF-8	AES, RSA	SHA-256

Key Takeaways

- ★ Use Encoding for data representation and compatibility.
- ★ Use Encryption to keep data confidential.
- ★ Use Hashing to verify data integrity or store passwords securely.
- ★ Choose the right tool for the right security requirement!

Remember

Confidentiality is about keeping data secret.
Integrity is about keeping data correct.
Encoding is just about formatting data.

11. JWT Fundamentals

What is JWT?

JWT (JSON Web Token) is a compact, URL-safe way to represent claims between two parties.

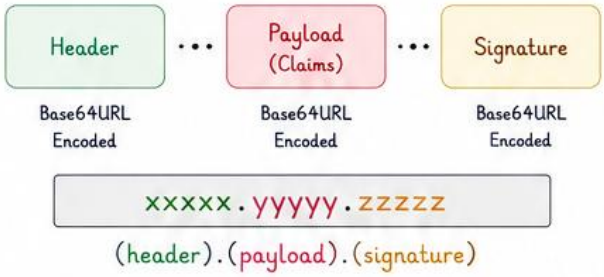
It is commonly used for authentication and information exchange.

JWTs are stateless and self-contained.

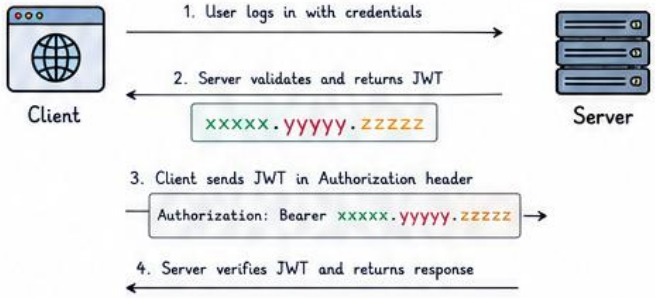
Key Points

- ✓ Compact and URL-safe
- ✓ Self-contained (contains claims)
- ✓ Digitally signed (integrity and authenticity)
- ✓ Stateless (no server-side session storage required)
- ✓ Can be used for authentication and authorization

Structure of JWT



How JWT Works (High Level)



JWT Components

Part	Description	Example
Header	Contains the type of token and the signing algorithm used.	`{ "alg": "HS256", "typ": "JWT" }`
Payload (Claims)	Contains the claims (information) about the user and other data.	`{ "sub": "1234567890", "name": "Nikhil", "role": "user", "iat": 1715683200, "exp": 1715686800 }`
Signature	Used to verify that the token is not modified and is from a trusted server.	`HMACSHA256( base64UrlEncode(header) + "." + base64UrlEncode(payload), secret_key)`

Common Claims

Claim	Description	Example
iss	Issuer (who issued the token)	"https://api.example.com"
sub	Subject (user identifier)	"1234567890"
aud	Audience (intended recipient)	"my-app"
exp	Expiration time (Unix timestamp)	1715686800
nbf	Not before (valid after time)	1715683200
iat	Issued at (time of issuance)	1715683200
jti	JWT ID (unique identifier)	"a1b2c3d4-e5f6..."

Standard claims are registered in the JWT specification.
You can also define custom claims.

Example (Decoded JWT)

Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ik5pa2hpbCIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzE1NjgzMjAwLCJleHAiOjE3MTU2ODY64MDB9.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

Header (Decoded)

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

→

Payload (Decoded)

```
{
  "sub": "1234567890",
  "name": "Nikhil",
  "role": "user",
  "iat": 1715683200,
  "exp": 1715686800
}
```

→

Signature

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret_key)
```

(Used to verify token)

Where JWT is Used?

- Authentication (Login)
- Authorization (Access Control)
- Mobile Apps
- APIs / Microservices
- Single Page Applications (SPA)
- Single Sign-On (SSO)
- Email Verification / Password Reset
- IoT / Device Authentication

Advantages

- ✓ Stateless – no server-side session storage required
- ✓ Self-contained – all necessary data inside the token
- ✓ Scalable – suitable for distributed systems
- ✓ Compact – small size, easy to transmit
- ✓ Secure – digitally signed to ensure integrity and authenticity

Limitations

- ✗ Cannot be revoked before expiration (unless using blacklist or short expiry)
- ✗ Do not store sensitive data in payload (it is only Base64 encoded, not encrypted)
- ✗ Token size limit (too many claims increase size)

Best Practices

- Use strong secret keys (HS256) or secure key pairs (RS256/ES256)
- Keep token expiry short
- Store tokens securely (HttpOnly cookies or secure storage)
- Validate token signature and claims on every request
- Do not trust data from client, always verify on server

Summary Comparison

Feature	Encoding	Encryption	Hashing	JWT
Purpose	Data representation	Data confidentiality	Data integrity	Authentication / Information Exchange
Reversible	Yes	Yes (with key)	No	Yes (paylood readable, not encrypted)
Key Required	No	Yes	No	Yes (for signature verification)
Output	Readable	Unreadable ciphertext	Fixed-size digest	Compact token (3 parts)
Example	Base64, UTF-8	AES, RSA	SHA-256	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... .yyyyy.zzzzz

12. JWT Structure + Claims

What is JWT?

JWT (JSON Web Token) is a compact, URL-safe way to represent claims between two parties.

It is used for authentication and information exchange.

JWT is stateless and self-contained.

Key Takeaways

- ✓ JWT is compact, URL-safe and self-contained.
- ✓ It consists of 3 parts: Header, Payload, Signature.
- ✓ Payload contains claims about the user and other data.
- ✓ Claims can be Standard (registered), Public (custom), or Private.
- ✓ Signature ensures integrity and authenticity.

1. JWT Structure

Header (Base64URL) . Payload / Claims (Base64URL) . Signature (Base64URL)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ik5pa2hpbCIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzE1NjgzMjAwLCJleHAiOjE3MTU2ODY4MDB9.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

- Each part is Base64URL encoded and separated by dots (.).
- The signature is created using the header, payload and a secret/private key (or private key in asymmetric algorithms).

2. Decoded JWT (Example)

Header (Decoded)

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

alg: algorithm used for signing (HS256)

typ: type of token (JWT)

Payload / Claims (Decoded)

```
{
  "sub": "1234567890",
  "name": "Nikhil",
  "role": "user",
  "iat": 1715683200,
  "exp": 1715686800,
  "iss": "your-app",
  "aud": "my-app"
}
```

Contains claims about the user and other data.

Signature (Decoded)

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret_key)
)
```

Used by server to verify token integrity.

3. Types of Claims

Type	Description	Example Claims	Example	Notes
1. Standard (Registered) Claims	Predefined claims that are not mandatory but recommended.	• iss (Issuer) • sub (Subject) • aud (Audience) • exp (Expiration Time) • nbf (Not Before) • iat (Issued At) • jti (JWT ID)	{ "iss": "https://api.example.com", "sub": "1234567890", "aud": "my-app", "exp": 1715686800, "nbf": 1715683200, "iat": 1715683200, "jti": "a1b2c3d4-e5f6-7890" }	Defined in the JWT spec (RFC 7519). Provide useful, interoperable information.
2. Public (Custom) Claims	Custom claims defined by you. Should be namespaced to avoid collisions.	• role • permissions • plan • department	{ "role": "admin", "permissions": ["read", "write"], "plan": "premium", "department": "engineering" }	Use a namespace like "https://your-app.com/role" to avoid collisions.
3. Private Claims	Custom claims agreed upon between parties. Not meant to be shared publicly.	• internal_id • feature_flags • app_metadata	{ "internal_id": "abc123", "feature_flags": ["beta_ui"], "app_metadata": {"tier": 2} }	Used internally between systems. Not part of any public namespace.

4. Common Standard Claims (Details)

Claim	Meaning	Example	Data Type
iss	Issuer (who issued the token)	"https://api.example.com"	String
sub	Subject (user identifier)	"1234567890"	String
aud	Auidence (intended recipient)	"my-app"	String / Array
exp	Expiration time (Unix timestamp)	1715686800	Number (int)
nbf	Not before (valid after this time)	1715683200	Number (int)
iat	Issued at (time of issuance)	1715683200	Number (int)
jti	JWT ID (unique identifier)	"a1b2c3d4-e5f6"	String

All time-based claims (exp, nbf, iat) use Unix timestamp (seconds).

5. How Claims are Used?

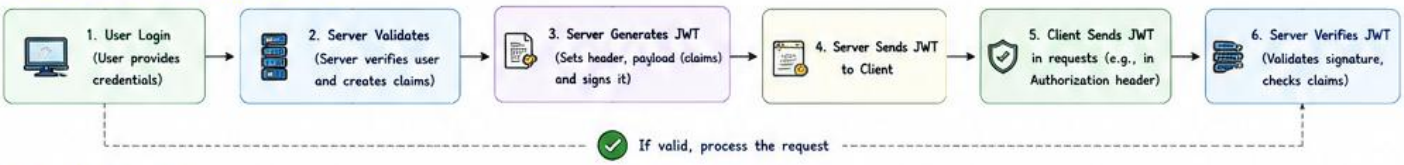
- **Authentication** — Verify the token and identify the user (sub)
- **Authorization** — Check roles, permissions, scopes, etc.
- **Session Management** — Use exp, nbf, iat to manage token validity
- **Personalization** — Use custom claims to customize user experience
- **Auditing & Tracking** — Use jti for token tracking and revocation

Note

Never store sensitive information in JWT payload (e.g., passwords, credit card, secrets).

JWT payload is Base64 encoded, not encrypted and can be decoded by anyone.

6. End-to-End Flow (High Level)



7. Quick Comparison

Feature	JWT	Notes
Format	Header.Payload.Signature (Base64URL)	Compact and URL-safe
Contains	Claims (user info, metadata, etc.)	Self-contained
Storage	Client side (e.g., localStorage, cookies)	No server-side session required
Verification	Signature verification	Ensures integrity and authenticity
Security	Use HTTPS, short expiry, secure storage	Do not store sensitive data in payload

Best Practices

- ✓ Use strong algorithms (HS256 / RS256 / ES256).
- ✓ Keep tokens short-lived (use refresh tokens if needed).
- ✓ Store tokens securely (HttpOnly cookies or secure storage).
- ✓ Always validate signature and claims on every request.
- ✓ Do not trust data from client; always verify on server.

13. JWT Signing + Verification

What is JWT Signing & Verification?

JWT (JSON Web Token) ensures authenticity and integrity using digital signatures.

- Signing: The server creates a signature using a secret key (HMAC) or private key (RSA/ECDSA).
- Verification: The server (or service) verifies the signature using the same secret key or the corresponding public key.

If the signature is valid → the token is trusted.

Key Takeaways

- ✓ JWT is signed to ensure it is from a trusted issuer and not tampered.
- ✓ Signing provides both authenticity and integrity.
- ✓ Verification must be done before trusting the claims.
- ✓ Use strong algorithms and secure key management.
- ✓ Never trust the payload before verification.

1. JWT Structure (Recap)

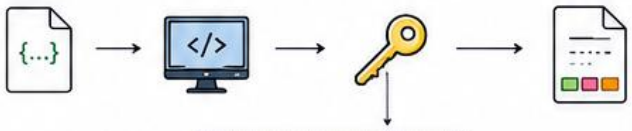
Header (Base64URL) . Payload / Claims (Base64URL) . Signature (Base64URL)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ik5pa2hpbCIsInJvbGUiOiJ1c2VyIn0.
Sf1KxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

```
Signature = HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload), secret_key)
```

2. How JWT Signing Works

1. Create Header & Payload → 2. Encode (Base64URL) → 3. Sign with Secret/Private Key → 4. Attach Signature to Form JWT



- Signing Algorithm
  - HMAC: HS256, HS384, HS512
  - RSA: RS256, RS384, RS512
  - ECDSA: ES256, ES384, ES512

3. JWT Signing (Example with HS256)

1. **Header**

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Base64URL → eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

2. **Payload (Claims)**

```
{
  "sub": "1234567890",
  "name": "Nikhil",
  "role": "user",
  "iat": 1715683200,
  "exp": 1715686800
}
```

Base64URL → eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ik5pa2hpbCIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzE1NjgzMjAwLCJleHAiOjE3MTU2ODY4MDB9

3. **Data to Sign**

```
base64UrlEncode(header) + "." + base64UrlEncode(payload)
```

4. **Signature**

HMACSHA256(data, secret_key) → Base64URL → Sf1KxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

5. **Final JWT**

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ik5pa2hpbCIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzE1NjgzMjAwLCJleHAiOjE3MTU2ODY4MDB9.Sf1KxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Note: Never share the secret key. Keep it secure on the server only.

4. JWT Verification Process

1. Receive JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
.....Sf1KxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

2. Split into 3 parts: Header . Payload . Signature
3. Recreate Signature

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret_key ) → Base64URL
```

4. Compare Signatures: Recreated Signature = Received Signature → Valid / Invalid
5. If valid, then process claims (check exp, iss, aud, etc.) Else, reject the token.

5. Algorithms for Signing

Algorithm Type	Examples	Key Used	Notes
HMAC (Symmetric)	HS256, HS384, HS512	Secret key	Fast, simple, same key for sign & verify.
RSA (Asymmetric)	RS256, RS384, RS512	Private key (sign) Public key (verify)	Slower than HMAC, but supports key separation.
ECDSA (Asymmetric)	ES256, ES384, ES512	Private key (sign) Public key (verify)	Shorter keys, faster than RSA.

6. Example (Verification with HS256)

Received JWT	Result
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9. eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6 Ik5pa2hpbCIsInJvbGUiOiJ1c2VyIiwiaWF0Ijox NzE1NjgzMjAwLCJleHAiOjE3MTU2ODY4MDB9. Sf1KxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c	If computed signature matches the last part → JWT is valid ✓ Else → Not valid ✗

7. Important Notes

- ✓ Always verify the signature before trusting the payload.
- ✓ Check standard claims: exp, nbf, iss, aud, sub, iat.
- ✓ Use HTTPS to transmit tokens.
- ✓ Store tokens securely (HttpOnly cookies/localStorage with caution).
- ✓ Use short-lived access tokens + refresh tokens.
- ✓ Rotate secrets/keys periodically.

8. Common Errors

- ✗ Using weak or hardcoded secret keys.
- ✗ Not verifying the signature.
- ✗ Accepting tokens with expired (exp) time.
- ✗ Ignoring audience (aud) or issuer (iss) validation.
- ✗ Storing tokens in insecure places (e.g., localStorage without protection).

9. Symmetric (HS256) vs Asymmetric (RS256) Comparison

Feature	HS256 (HMAC)	RS256 (RSA)
Key Type	Symmetric (one secret key)	Asymmetric (key pair)
Signing Key	Secret key	Private key
Verification Key	Same secret key	Public key
Performance	Faster	Slower
Use Case	Internal services, same trust domain	Public APIs, third-party verification
Key Distribution	Share secret securely	Share public key (safe), keep private key secret
Security	If secret leaks, both sign & verify compromised	Private key remains secret; safer for distributed systems

Quick Checklist

- ✓ Header correct?
- ✓ Claims valid?
- ✓ Signature verified?
- ✓ Token not expired?
- ✓ Issuer & Audience valid?

If all ✓ → Token is trusted!

14. JWT Authentication + Security

1. What is JWT Authentication?

JWT Authentication is a stateless way to authenticate users.

After successful login, the server issues a JWT.

The client sends this token in subsequent requests.

The server verifies the token and grants access.

No server-side session storage required.

Key Takeaways

- ✓ JWT enables stateless authentication.
- ✓ Token is self-contained and signed.
- ✓ Always verify the token before trusting any claims.
- ✓ Keep tokens short-lived and secure.
- ✓ Use HTTPS and follow best security practices.

2. JWT Authentication Flow (High Level)



3. Where to Store the JWT?

Storage Option	How to Use	Pros	Cons	Recommended?
Memory (JS variable)	Store in memory only	Least vulnerable to XSS	Lost on page refresh	✓ Best (Web)
HTTP-Only Cookie	Store in cookie (HttpOnly, Secure, SameSite)	Protected from XSS	Vulnerable to CSRF	✓ Good
localStorage	Store in localStorage	Persistent	Vulnerable to XSS	✗ Not Recommended
sessionStorage	Store in sessionStorage	Cleared on tab close	Vulnerable to XSS	✗ Not Recommended

4. Sending JWT in Requests

Authorization Header (Recommended)	Other Ways (Not Recommended)
Include the token in the Authorization header.	• Query Parameters (?token=...)
`Authorization: Bearer <JWT_TOKEN>`	• Request Body
	⚠ Avoid these. Tokens can leak in logs, URLs, caches.

5. Token Validation (What Server Should Do)

- ✓ Verify Signature – Ensure the token is signed with the correct secret/public key.
- ✓ Check Expiration (exp) – Reject if token is expired.
- ✓ Check Issuer (iss) – Ensure token is issued by your server.
- ✓ Check Audience (aud) – Ensure token is intended for your application.
- ✓ Check Not Before (nbf) – Ensure token is not used before valid time.
- ✓ Validate Subject (sub) – Ensure the user exists and is active (if needed).
- ✓ Validate Custom Claims – Roles, permissions, scopes, etc.

6. Token Expiration & Refresh Tokens

Access Token (Short-Lived)	Refresh Token (Long-Lived)
• Short expiry (5 – 15 min)	• Long expiry (days/weeks)
• Used to access APIs	• Used to get a new access token
• Limits damage if token is stolen	• Store securely (HttpOnly cookie)
	• Rotate refresh tokens

Refresh Flow



7. Standard Claims for Security

Claim	Purpose	Security Usage
iss (Issuer)	Who issued the token	Ensure token comes from your server
aud (Audience)	For whom the token is intended	Prevent token misuse on other apps
exp (Expiration)	When the token expires	Reject expired tokens
nbf (Not Before)	When token becomes valid	Prevent premature use
iat (Issued At)	When the token was issued	For logging, debugging, anomaly detection
jti (JWT ID)	Unique ID for the token	Prevent replay attacks (use blacklist)

8. Common Security Threats & Mitigations

Threat	Description	Mitigations
XSS (Cross-Site Scripting)	Steals token from localStorage or JS accessible storage	Use HttpOnly cookies or in-memory storage
CSRF (Cross-Site Request Forgery)	Attacker sends requests using victim's cookie	Use SameSite cookies, CSRF tokens, or double submit
Token Theft	Token intercepted in transit	Always use HTTPS
Replay Attacks	Stolen token reused	Short-lived tokens, jti claim, rotate refresh tokens
Weak Secrets	Easy to guess secret keys	Use strong, random secrets (> 32 chars)
Algorithm Confusion	Changing alg to 'none' or weak alg	Allow only expected algorithms in verification

9. Best Practices

- ✓ Use HTTPS everywhere.
- ✓ Keep access tokens short-lived.
- ✓ Store tokens securely (HttpOnly cookies or memory).
- ✓ Validate all standard claims.
- ✓ Use strong signing algorithms (HS256, RS256, ES256).
- ✓ Never store sensitive data in JWT payload.
- ✓ Use refresh token rotation.
- ✓ Revoke tokens on logout (blacklist or versioning).
- ✓ Monitor and log suspicious activities.
- ✓ Limit token size (JWT should be compact).

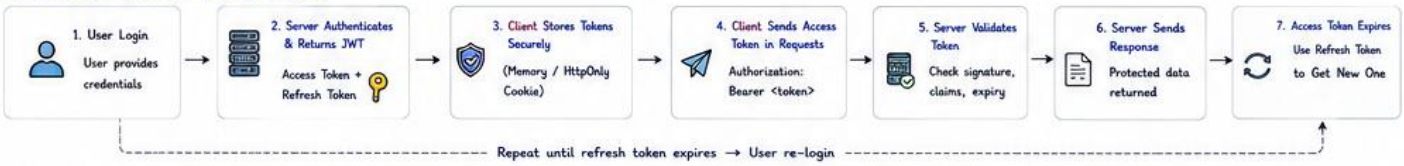


10. Logout & Token Revocation Strategies

1. Blacklist (Deny List)	2. Token Versioning	3. Short Expiry + Refresh (Token Rotation)
Store revoked token IDs (jti) in database or cache (Redis).	Store user token version in DB. Include in token claim (ver). On logout, increase version.	Keep access token short-lived. Use refresh token rotation and revoke old ones.
Check blacklist on every request.	ver: 3 → ver: 4	
	Older tokens become invalid.	

💡 Choose strategy based on your scale and requirements.

11. Example (End-to-End Overview)



12. Do's and Don'ts

✓ Do's

- Do use strong secrets and algorithms.
- Do use HTTPS.
- Do validate tokens on every request.
- Do keep tokens short-lived.
- Do secure refresh tokens.
- Do handle logout and revocation.
- Do monitor and audit.

✗ Don'ts

- Don't store tokens in localStorage.
- Don't use weak or default secrets.
- Don't trust token without verification.
- Don't store sensitive data in payload.
- Don't use long-lived access tokens.
- Don't ignore token revocation.
- Don't expose tokens in URLs or logs.

13. Quick Checklist

- ✓ Is HTTPS enabled?
- ✓ Are tokens signed with a strong algorithm?
- ✓ Are all claims validated (exp, iss, aud, nbf, etc.)?
- ✓ Is token stored securely on client?
- ✓ Is CSRF protection in place (if using cookies)?
- ✓ Are refresh tokens rotated and secured?
- ✓ Is logout revocation handled?
- ✓ Are secrets stored safely (env variables / secret manager)?

15. ENCRYPTION + JWT QUICK REVISION NOTES

A. ENCRYPTION BASICS

- Encryption converts readable data (plaintext) into unreadable data (ciphertext) using an algorithm and a key.
- It ensures Confidentiality.
- Decryption with the correct key converts ciphertext back to plaintext.

Flow
Plaintext ↓ Encrypt ↓ Ciphertext ↓ Decrypt ↓ Plaintext

B. TYPES OF ENCRYPTION

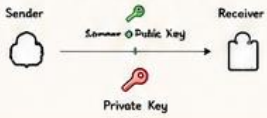
1. Symmetric Encryption

- Same key for encryption & decryption.
- Fast, ideal for large data.
- Key must be shared securely.
- Examples: AES, DES, 3DES, ChaCha20



2. Asymmetric Encryption

- Uses a key pair: Public Key & Private Key.
- Slower, ideal for key exchange & authentication.
- Examples: RSA, ECC, ElGamal



C. COMMON ENCRYPTION ALGORITHMS

Algorithm	Type	Key Size	Use	Notes
AES (Advanced Encryption Standard)	Symmetric	128 / 192 / 256 bits	General purpose encryption	Most secure & widely used
DES (Data Encryption Standard)	Symmetric	56 bits	Legacy systems	Insecure now
3DES (Triple DES)	Symmetric	112 / 168 bits	Legacy systems	Slower, use AES
RSA	Asymmetric	2048 / 3072 / 4096 bits	Key exchange, Digital signatures	Slower, secure
ECC (Elliptic Curve Cryptography)	Asymmetric	224 / 256 / 384 bits	Key exchange, Signatures	Faster & smaller keys than RSA

D. HASHING (ONE-WAY)

- Converts data into fixed-size digest (one-way).
- Used for integrity verification, password storage.
- Examples: SHA-256, SHA-512, Bcrypt, Argon2

Input ↓ Hash Function ↓ Hash (Digest)

E. KEYS & KEY MANAGEMENT

Key Types

- Symmetric Key (Secret Key) Same key for enc/dec.
- Public Key (Shared openly)
- Private Key (Kept secret)
- Session Key (Temporary key for a session)

Best Practices

- ✓ Use strong keys (sufficient length).
- ✓ Store keys securely (KMS, HSM, Vault).
- ✓ Rotate keys regularly.
- ✓ Limit access (least privilege).
- ✓ Never hardcode keys in code.
- ✓ Use secure channels for key exchange.

F. ENCODING vs ENCRYPTION vs HASHING

Feature	Encoding	Encryption	Hashing
Purpose	Data Representation	Confidentiality	Data Integrity
Reversible?	Yes	Yes (with key)	No
Uses Key?	No	Yes	No
Output	Readable	Unreadable (Ciphertext)	Fixed-size Digest
Example	Base64, UTF-8	AES, RSA	SHA-256

ENCRYPTION CHEAT SHEET

- ✓ Want speed & large data security? → Use Symmetric (AES)
- ✓ Need secure key exchange / authentication? → Use Asymmetric (RSA/ECC)
- ✓ Need password storage / integrity? → Use Hashing (Bcrypt, Argon2, SHA-256)
- ✓ Need both confidentiality & integrity? → Use Encryption + HMAC / Digital Signatures

G. JWT (JSON WEB TOKEN) OVERVIEW

- JWT is a compact, URL-safe way to represent claims between two parties.
- Used for Authentication and Information Exchange.
- It is Stateless, Self-contained and Signed.

H. JWT STRUCTURE

Header (Base64URL) . Payload / Claims (Base64URL) . Signature (Base64URL)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 .
eyJzdWIiOiIxMjMiLCJyb2xéIjoiYWRtaW4iLCJleHAiOjE3MTU5ODgwMDB9 .
Sf1KxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

I. JWT COMPONENTS

Part	Description	Example (Decoded)
Header	Contains type of token and signing algorithm.	{ "alg": "HS256", "typ": "JWT" }
Payload (Claims)	Contains claims about user and other data.	{ "sub": "1234567890", "name": "Nikhil", "role": "user", "exp": 1715686800 }
Signature	Created using Header, Payload and Secret/Private Key.	HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret_key)

J. COMMON JWT CLAIMS

Claim	Meaning	Example
iss (Issuer)	Issuer of the token	"https://api.example.com"
sub (Subject)	Subject (user id)	"1234567890"
aud (Audience)	Intended audience	"my-app"
exp (Expiration Time)	Token expiration time (Unix)	1715686800
nbf (Not Before)	Token valid after this time	1715683200
iat (Issued At)	Token issued at (Unix)	1715683200
jti (JWT ID)	Unique token ID	"a1b2c3d4-e5f6..."

K. JWT SIGNING & VERIFICATION

Signing (By Server)

1. Create Header & Payload
2. Encode both using Base64URL
3. Create Signature using Secret/Private Key
   Signature = HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret_key)
4. Append signature to form JWT

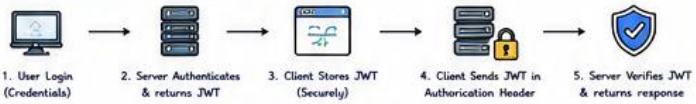
Header + Payload + Signature

Verification (By Client / Server)

1. Receive JWT
2. Split into 3 parts (Header, Payload, Signature)
3. Recreate signature using same algorithm and secret/public key
4. Compare recreated signature with received signature
5. If match → Token is valid
   Else → Invalid

Recreated Signature = Received Signature → Valid / Invalid

L. JWT AUTHENTICATION FLOW



Authorization Header: Bearer <JWT_TOKEN>

M. JWT SECURITY BEST PRACTICES

- ✓ Use HTTPS everywhere.
- ✓ Use strong signing algorithms (HS256, RS256, ES256).
- ✓ Keep tokens short-lived (5–15 min).
- ✓ Use Refresh Tokens for long sessions.
- ✓ Store tokens securely (HttpOnly cookies / memory).
- ✓ Never store sensitive data in payload.
- ✓ Validate all standard claims.
- ✓ Implement logout (blacklist / token versioning).
- ✓ Rotate secrets / keys regularly.
- ✓ Limit token size (JWT should be compact).

QUICK COMPARISON: ENCRYPTION vs JWT

Encryption	JWT
✓ Protects data (Confidentiality)	✓ Token for Authentication & Claims
✓ Can be symmetric or asymmetric	✓ Self-contained & Signed (not encrypted)
✓ Requires decryption to read	✓ Decoded to read claims (not for secrets)
✓ Used for data at rest / in transit	✓ Used for stateless sessions / APIs
✓ Examples: AES, RSA	✓ Example: JWT with HS256 / RS256

REMEMBER

- Encryption → Keep data secret
- Hashing → Ensure data integrity
- JWT → Verify identity & share claims

"Secure the data. Verify the user. Trust nothing, verify everything."

MINI CHECKLIST

- ☐ Understand Symmetric vs Asymmetric encryption
- ☐ Know AES, RSA and hashing basics
- ☐ Understand JWT structure & claims
- ☐ Know how signing & verification works
- ☐ Follow JWT security best practices